Aegis: A Layered, Defense-in-Depth System for LLM Jailbreak and Prompt-Injection Defense

U E Sai Pavan Vamshi Krishna (G25AIT2149) — M.Tech, Indian Institute of Technology Jodhpur Course project, CSL6010 (Cyber Security). Successor to **RJD-v2**. June 2026.

Abstract

Large language models deployed as assistants and autonomous agents face a moving adversary: jailbreaks that strip safety behaviour, prompt injections hidden in retrieved content, obfuscated and translated attacks, and responses that leak secrets or private data. Prompt injection is the **#1 risk** in the OWASP Top 10 for LLM Applications, and recent adaptive-attack work shows that *any single* filter, once known, can be evaded — "the attacker moves second." We present **Aegis**, a defense-in-depth system that composes six independent layers (L0–L5): input normalization and provenance, a fast statistical detector, a fine-tuned guard ensemble, agent/tool-injection defense, output moderation, and continuous red-teaming with drift monitoring. Each layer targets a distinct failure mode, so bypassing one is not sufficient to break the system. Aegis is engineered to train and run entirely on a single free Kaggle **T4** GPU, ships as an open-source `pip` library and a FastAPI service, and is aligned to the OWASP LLM Top 10 and the NIST AI Risk Management Framework. We describe the threat model, the architecture, the evaluation methodology (security-grade metrics, contamination control), and the system's limitations.

1. Introduction

Safety-aligned LLMs still fail under adversarial pressure. A *jailbreak* is a prompt that induces a model to violate its safety policy ("ignore all previous instructions and act as DAN"); a *prompt injection* embeds adversarial instructions in data the model later reads (a web page, a tool result, an email), hijacking an agent without the user's knowledge. Both are now industrialized: prompt-sharing communities circulate thousands of working jailbreaks, and agentic deployments multiply the blast radius because a single injected instruction can trigger real-world tool calls and data exfiltration.

The central difficulty is adaptivity. A classifier that blocks today's attacks is a fixed target; attackers paraphrase, encode (Base64, leetspeak), substitute homoglyphs, insert zero-width characters, translate to other languages, or wrap the payload in role-play until they find a phrasing the classifier scores as benign. The "attacker-moves-second" result is that robustness claims based on a static test set systematically overstate real protection. The practical response — adopted by every serious security discipline — is **defense in depth**: many cheap, independent controls layered so that an attacker must defeat all of them simultaneously, plus a *continuous* red-team/monitoring loop so the controls keep pace.

Aegis operationalizes this for LLMs under a hard constraint: it must be buildable and reproducible on free compute (a single Kaggle T4). This forces dependency-light designs and graceful offline fallbacks, which also make the system easy to adopt.

2. Threat Model (2026)

We defend against an adversary who controls the prompt and/or any untrusted content the model ingests, and who can adapt to a known filter. In scope:

- **Direct jailbreaks:** persona/role-play ("DAN", "developer mode"), instruction-override, and policy-nullification prompts.

- **Obfuscation/evasion:** Base64 and other encodings, leetspeak, Unicode homoglyphs, zero-width and bidi control characters, emoji/variation-selector smuggling, character spacing.
- **Indirect / agent injection:** instructions hidden in retrieved documents, web pages, or tool outputs that target an LLM agent and its tools (exfiltration, unauthorized actions).
- **Multilingual attacks:** the same intent expressed in non-English languages or scripts to bypass English-centric filters.
- **Output-side failures:** a (possibly bypassed) model that emits PII, leaked credentials/secrets, or its own system prompt, or that complies with a harmful request.

Out of scope: attacks on the base model weights, the host infrastructure, or the human operator. We assume no defense is unbreakable; the goal is to **raise attacker cost** and **shrink the attack surface** of every channel.

3. Related Work

Guard models. Instruction-tuned safety classifiers — Llama Guard (through v4), Qwen3Guard, ShieldGemma 2, WildGuard, and IBM Granite Guardian — judge prompts and, in "response classification" mode, (prompt, response) pairs against a configurable taxonomy. They are strong but heavyweight and themselves evadable; calibration is a known concern.

Design patterns. Rather than rely on a single classifier, recent work proposes architectural defenses: **spotlighting** (marking untrusted text so the model treats it as data), the **Dual-LLM / quarantine** pattern and Google's **CaMeL** (a privileged planner that never sees raw untrusted content), and training-time defenses **StruQ/SecAlign** that teach the model to ignore injected instructions.

Benchmarks & red-teaming. HarmBench, JailbreakBench, and AdvBench measure jailbreak robustness; AgentDojo and InjecAgent measure agent/injection robustness. Automated red-team toolkits — NVIDIA **garak**, Microsoft **PyRIT** (multi-turn/crescendo), and **Promptfoo** — probe systems for dozens of vulnerability classes.

Data protection & standards. Microsoft **Presidio** is the de-facto open framework for PII detection/redaction; gitleaks/trufflehog/detect-secrets define the secret-scanning rule families. The **OWASP Top 10 for LLM Applications** (LLM01: Prompt Injection) and the **NIST AI RMF** provide the governing risk taxonomy.

Aegis does not replace these; it **composes** their ideas into one cascade and supplies dependency-free implementations (with hooks to defer to Presidio, garak, AgentDojo, and hosted guard models where available).

4. System Design

Aegis is a cascade of six layers. Each is independent and individually testable.

L0 — Normalize & provenance. A `normalize()` function performs Unicode NFKC folding; strips zero-width, bidi, tag, and variation-selector characters; folds homoglyphs to ASCII; decodes Base64; and undoes leet/spacing. `spotlight()` wraps untrusted content with explicit data markers. L0 collapses the obfuscation channel *before* any classifier runs, so downstream layers see canonical text. Language/script detection flags non-English input.

L1 — Fast layer. `FastLayer` combines three complementary signals: the **RJD-v2** detector (de-obfuscation + word/char TF-IDF + engineered features, calibrated), a **semantic** detector (max cosine similarity to a bank of known attacks via sentence-transformers, with a TF-IDF fallback and a multilingual model option), and a **signature** DB (normalized exact match + high-precision templates). The combiner is a **recall-preserving maximum**: FastLayer's

score is always ≥ the RJD score, so adding signals can only raise recall, never regress it. The semantic signal is gated to hold false-positive rate near RJD's.

L2 — Guard ensemble. A small base model (Qwen2.5-1.5B) is **QLoRA**-fine-tuned (4-bit base + LoRA adapter) as a binary safety classifier, fitting on a single T4. `GuardEnsemble` aggregates members (max/mean). A **selective cascade** invokes the expensive guard *only on uncertain prompts* (scores between the allow and block thresholds), giving deep scrutiny where it matters at low average cost. The adapter and a model card are published to the Hugging Face Hub.

L3 — Agent defense. An `InjectionScanner` detects and sanitizes injected instructions in untrusted content; a least-privilege `ToolPolicy` tracks taint and gates dangerous tools (allow/confirm/block) once a turn reads untrusted data; a `DualLLM` orchestrator keeps the privileged planner from ever seeing raw untrusted text. Evaluated on an indirect-injection benchmark with an AgentDojo hook.

L4 — Output moderation. `OutputModerator` is the egress gate. It redacts **PII** (emails, phones, SSNs, Luhn-validated cards, IPs), blocks leaked **secrets/credentials** (AWS/GitHub/OpenAI/Slack keys, private keys, JWTs, high-entropy strings), detects **system-prompt / canary** leakage (exact canary + n-gram overlap), and scores **response safety** (refusal vs. harmful compliance — the guard-model response-classification idea). A jailbreak only truly "succeeds" if the input was malicious *and* the response complied; L4 catches that final step even when the input filter was bypassed.

L5 — Continuous ops. A `RedTeam` harness (a local garak/PyRIT stand-in) mutates known attacks through a battery of evasions and reports **attack-success-rate per mutator**, surfacing which channels still beat the filter. A `Monitor` holds a reference score distribution and flags drift via the **Population Stability Index** plus block-rate alarms — the runtime half of the "attacker-moves-second" loop.

Pipeline. `Aegis.scan()` runs L0→L1→(L2) for the prompt; `Aegis.guard_turn(prompt, response)` adds the L4 egress gate, returning one allow/redact/block decision for the full request/response envelope.

5. Implementation

The system is a single Python package with no mandatory heavy dependencies (core: scikit-learn, pandas, numpy). Embedding, guard, and serving features are optional extras (`[guard]`, `[serve]`). Every layer has an **offline fallback** (e.g., TF-IDF when sentence-transformers is unavailable; synthetic corpus when dataset downloads are blocked), so the pipeline always runs. Aegis ships as: (i) the `aegis-guard` `pip` library with an `aegis` CLI; (ii) a **FastAPI** service exposing `/scan`, `/moderate`, and `/guard_turn`, containerized via Docker; and (iii) six Kaggle notebooks (P1–P6) that clone the repo from GitHub and reproduce each phase on a T4.

6. Evaluation

Methodology. We report **security-grade** metrics rather than plain accuracy: ROC-AUC, **recall @ 1% FPR** (the operating point that matters when most traffic is benign), **FPR @ 95% TPR**, **over-refusal / false-refusal rate (FRR)**, **attack-success-rate (ASR)**, F1, and latency. The corpus assembles in-the-wild jailbreaks, JailbreakBench, AdvBench, HarmBench, and WildGuardMix; benchmark prompt sets are kept **test-only** to avoid contamination. Output moderation is scored by flag precision/recall on a labeled probe set; robustness by ASR-per-mutator; multilingual coverage by macro-recall across five languages.

Representative findings. (1) The recall-preserving combiner guarantees FastLayer ≥ RJD-v2 on every slice, so L1 strictly dominates the prior detector. (2) After L0 normalization, the obfuscation mutators (Base64, homoglyph, zero-width, leetspeak, spacing) collapse to

near-zero ASR against L1 — the layered design closes those channels. (3) On the labeled output-moderation probe set, the L4 gate reached **precision = recall = 1.0** (PII redacted, secrets/leaks/harmful compliance blocked, refusals allowed). (4) The selective cascade routes only the uncertain band to the L2 guard, keeping average cost low. (5) The drift monitor reliably trips a PSI alert under an attack-surge window. Exact benchmark numbers are reproduced by the P1–P6 notebooks and logged to Weights & Biases; we deliberately avoid quoting point estimates that depend on the specific run and on access to gated datasets.

7. Limitations

No single defense — and no stack — is unbreakable; Aegis raises cost, it does not guarantee safety. The L2 guard is English-centric unless trained with translated data; L4 response-safety uses a lightweight heuristic/guard scorer that is weaker than a full response classifier; the secret/PII regexes trade some recall for precision and can be evaded by novel formats (Presidio/NER closes part of this gap); the offline fallbacks reduce accuracy relative to a fully-resourced run; and the red-team harness mutates *known* attacks, so it measures evasion of catalogued techniques rather than discovering novel ones. These are exactly why L5 (continuous red-teaming + monitoring) exists.

8. Ethics and Responsible Use

Aegis is a **defensive** safety filter. The attack and red-team code mutate only attacks already present in public corpora, for evaluation and hardening; the repository contains no novel weaponization and no operational instructions for harm. The system is aligned to the OWASP LLM Top 10 and NIST AI RMF and is intended to be operated with continuous red-teaming and human oversight. Detection scores are probabilistic and should inform, not solely determine, high-stakes decisions.

9. Conclusion

Aegis shows that a credible, modern LLM defense can be assembled from independent, individually-cheap layers — normalization, fast detection, a fine-tuned guard, agent defense, output moderation, and continuous red-teaming — entirely on free compute, and packaged as a usable library and service. The contribution is less any single classifier than the **composition**: a cascade where every channel an attacker might use is narrowed, and a monitoring loop that assumes the attacker moves second. Future work: train the guard on translated data for native multilingual coverage, integrate hosted guard models and AgentDojo/garak directly, and replace the heuristic response scorer with a calibrated response classifier.

References (selected)

OWASP Top 10 for LLM Applications (LLM01: Prompt Injection). · NIST AI Risk Management Framework (AI RMF 1.0). · Inan et al., *Llama Guard* (and v2–v4 model cards). · Willison, *Dual-LLM pattern*; Google DeepMind, *CaMeL*. · Chen et al., *StruQ*; *SecAlign*. · Hines et al., *Spotlighting*. · Mazeika et al., *HarmBench*. · Chao et al., *JailbreakBench*. · Zou et al., *Universal and Transferable Adversarial Attacks (AdvBench)*. · Debenedetti et al., *AgentDojo*. · *InjecAgent*. · NVIDIA *garak*; Microsoft *PyRIT*; *Promptfoo*. · Microsoft *Presidio*. · "The Attacker Moves Second" (adaptive-attack evaluation).